

DNS Resolver with Caching, Security, and Logging

Project Report

Submitted By

Muhammad Abser | 23K0030

Muhammad Ali | 23K0052

FAST School of Computing

National University of Computer & Emerging Sciences

Abstract

This project implements a DNS forwarding system with TTL-aware caching, rate limiting, query randomization, and DNSSEC-aware response checks. It is designed to demonstrate practical resolver behavior, measurable performance benefits from caching, and basic defenses against cache poisoning and request flooding.

1. Motivation

The project was selected to study how a practical DNS resolver can be improved with caching and basic security controls while remaining simple enough to build, test, and explain end to end.

2. Overview

2.1 Significance of the Project

This project is significant because it combines resolver behavior, cache design, logging, and security hardening in one system. It is practical because it runs as a local UDP service and can be benchmarked directly.

2.2 Description of the Project

The system is a DNS forwarding server written in Python. It listens for client queries, forwards them to an upstream resolver, stores responses in an in-memory TTL cache, and returns cached answers when valid entries already exist.

2.3 Background of the Project

The design is informed by common DNS implementations and resolver security practices. BIND9 and dnspython provide the basis for standard DNS handling, while DNSSEC references describe authenticity and integrity validation.

2.4 Project Category

This is a product-based implementation project with a strong research and evaluation component.

3. Features / Scope / Modules

1. Recursive DNS forwarding with upstream query handling and structured response parsing.
2. TTL-aware cache that improves repeated-query latency and expires stale entries automatically.
3. Per-client rate limiting to reduce abuse and deny simple flooding behavior.
4. Cache-poisoning hardening using 0x20 randomized casing and randomized source ports.
5. DNSSEC-aware validation checks for signed data, helping avoid blind trust in unsigned answers.
6. CSV logging that records query name, type, cache hit status, response code, and latency.
7. Benchmark mode that compares cold and warm lookups to quantify caching gains.

4. Project Planning

Week 1: environment setup and DNS library exploration. Week 2: query forwarding and response parsing. Week 3: TTL caching and benchmarking. Week 4: security checks and rate limiting. Week 5: integration testing and report preparation.

Team responsibilities:

- Muhammad Abser (23K0030): cache-poisoning defenses, rate limiting, DNSSEC-aware checks, report integration.
- Muhammad Ali (23K0052): query forwarding, caching, logging, benchmarking, and validation runs.

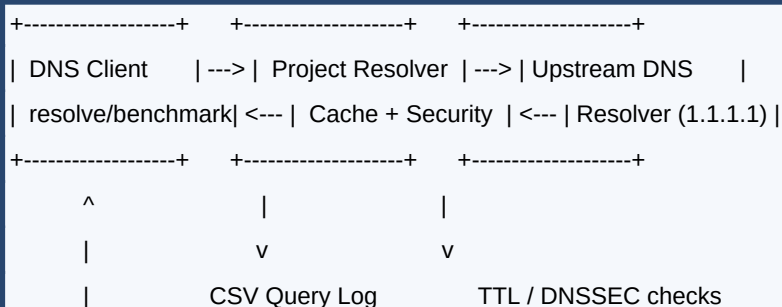
5. Project Feasibility

- Technical feasibility: the project is implementable with standard Python networking libraries and a public upstream DNS resolver.
- Economic feasibility: the software is low-cost and runs on a normal laptop or lab PC without special hardware.
- Schedule feasibility: the scope is small enough to complete in a short academic timeline while still supporting a meaningful demonstration.

6. Hardware and Software Requirements

- Hardware: a standard PC or laptop with network access.
- Operating system: Windows, Linux, or macOS; the implementation was tested in a Windows workspace.
- Software: Python 3.13, dnspython, and CSV-compatible tooling for log analysis.
- Optional tools: a local packet sniffer or Wireshark for observing query behavior.

7. Diagrammatic Representation of the Overall System



8. Results from Implementation

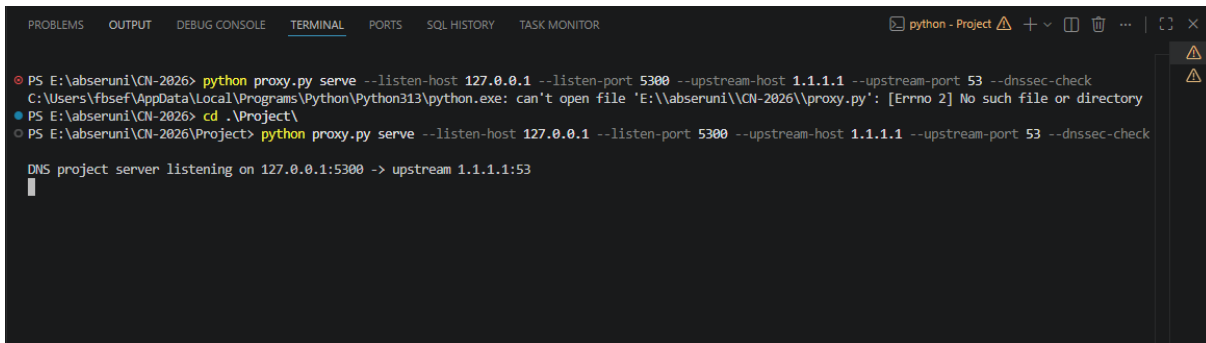
The current query log contains 39 recorded DNS transactions. Cache hits account for 28 queries (71.8%), while cache misses account for 11 queries.

The average latency of cached responses is 0.993 ms, compared with 151.881 ms for cache misses. The observed latency range is 0.621 ms to 529.749 ms.

- No REFUSED responses were recorded in the available log slice.
- No SERVFAIL responses were recorded in the available log slice.
- Most frequently queried name: example.com..
- Most frequently queried type: A.

9. Working Demo Screenshots - Server and Benchmark

Server startup and listener configuration



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SQL HISTORY  TASK MONITOR
python - Project

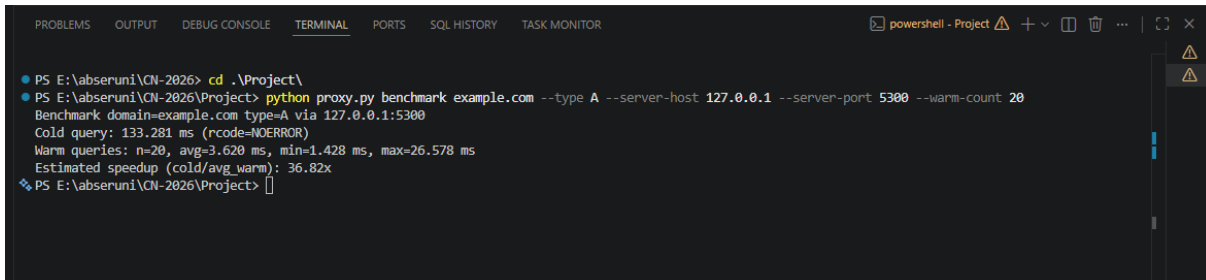
PS E:\abseruni\CN-2026> python proxy.py serve --listen-host 127.0.0.1 --listen-port 5300 --upstream-host 1.1.1.1 --upstream-port 53 --dnssec-check
C:\Users\fbsef\AppData\Local\Programs\Python\Python313\python.exe: can't open file 'E:\abseruni\CN-2026\proxy.py': [Errno 2] No such file or directory
PS E:\abseruni\CN-2026> cd .\Project\
PS E:\abseruni\CN-2026\Project> python proxy.py serve --listen-host 127.0.0.1 --listen-port 5300 --upstream-host 1.1.1.1 --upstream-port 53 --dnssec-check

DNS project server listening on 127.0.0.1:5300 -> upstream 1.1.1.1:53

```

- The server starts on 127.0.0.1:5300 and forwards queries to the configured upstream resolver.
- This confirms the resolver is available before any client query is issued.

Benchmark mode showing cold-vs-warm cache performance



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SQL HISTORY  TASK MONITOR
powershell - Project

PS E:\abseruni\CN-2026> cd .\Project\
PS E:\abseruni\CN-2026\Project> python proxy.py benchmark example.com --type A --server-host 127.0.0.1 --server-port 5300 --warm-count 20
Benchmark domain=example.com type=A via 127.0.0.1:5300
Cold query: 133.281 ms (rcode=NOERROR)
Warm queries: n=20, avg=3.620 ms, min=1.428 ms, max=26.578 ms
Estimated speedup (cold/avg_warm): 36.82x
PS E:\abseruni\CN-2026\Project>

```

- The benchmark compares the first uncached lookup against repeated cached lookups.
- The observed speedup supports the cache effectiveness claim in the report.

9.1 Working Demo Screenshots - Resolution Output

Forward resolution examples for A, NS, MX, and AAAA queries

```

PS E:\abseruni\CN-2026\Project> python proxy.py resolve www.youtube.com --type A --server-host 127.0.0.1 --server-port 5300
Query time: 144.127 ms
rcode=NOERROR
[ANSWER]
  www.youtube.com. 300 IN CNAME youtube-ui.l.google.com.
  youtube-ui.l.google.com. 300 IN A 142.251.142.78
youtube-ui.l.google.com. 300 IN A 142.251.288.238
youtube-ui.l.google.com. 300 IN A 142.250.181.206
youtube-ui.l.google.com. 300 IN A 142.251.36.142
youtube-ui.l.google.com. 300 IN A 142.251.209.110
youtube-ui.l.google.com. 300 IN A 142.251.209.174
youtube-ui.l.google.com. 300 IN A 142.251.209.78
youtube-ui.l.google.com. 300 IN A 142.250.178.14
youtube-ui.l.google.com. 300 IN A 142.251.36.46
youtube-ui.l.google.com. 300 IN A 172.217.20.110
youtube-ui.l.google.com. 300 IN A 172.217.18.110
youtube-ui.l.google.com. 300 IN A 172.217.17.46
youtube-ui.l.google.com. 300 IN A 142.251.39.174
youtube-ui.l.google.com. 300 IN A 142.250.181.110
youtube-ui.l.google.com. 300 IN A 142.251.36.206
youtube-ui.l.google.com. 300 IN A 142.251.288.142
[AUTHORITY]
<empty>
[ADDITIONAL]
<empty>
PS E:\abseruni\CN-2026\Project> python proxy.py resolve www.youtube.com --type NS --server-host 127.0.0.1 --server-port 5300
Query time: 134.210 ms
rcode=NOERROR
[ANSWER]
  www.youtube.com. 300 IN CNAME youtube-ui.l.google.com.
[AUTHORITY]
  l.google.com. 60 IN SOA ns1.google.com. dns-admin.google.com. 912537324 900 900 1800 60
[ADDITIONAL]
<empty>
PS E:\abseruni\CN-2026\Project> python proxy.py resolve youtube.com --type MX --server-host 127.0.0.1 --server-port 5300
Query time: 126.288 ms
rcode=NOERROR
[ANSWER]
  youtube.com. 300 IN MX 0 smtp.google.com.
[AUTHORITY]
<empty>
[ADDITIONAL]
<empty>
PS E:\abseruni\CN-2026\Project> python proxy.py resolve youtube.com --type AAAA --server-host 127.0.0.1 --server-port 5300
Query time: 124.108 ms
rcode=NOERROR
[ANSWER]
  youtube.com. 300 IN AAAA 2a00:1450:4006:818::200e
[AUTHORITY]
<empty>
[ADDITIONAL]
<empty>
PS E:\abseruni\CN-2026\Project>

```

10. References

1. RFC 4033, DNS Security Introduction and Requirements.
2. BIND 9 Administrator Reference Manual.
3. dnspython documentation.
4. RFC 5358, Preventing DNS-Induced Denial-of-Service Attacks.
5. Kaminsky, D. (2008), DNS cache poisoning and related resolver attack discussions.
6. Ubuntu DNS documentation and resolver deployment guides.

11. Conclusion

The project demonstrates a functional DNS resolver prototype with measurable caching benefits and multiple defensive mechanisms. It is suitable as an academic project because the implementation is small enough to understand, yet rich enough to show real networking, security, and performance concepts in practice.